Word Embeddings and Character-Level Name Generation

CSL7640 – Natural Language Understanding
Assignment 2

Agam Harpreet Singh
B23CM1004

March 2026

Abstract

This report covers two related problems in representation learning and sequence modelling. In Problem 1, we collect a text corpus from the IIT Jodhpur website and train Word2Vec models (CBOW and Skip-gram with Negative Sampling) from scratch using PyTorch. We perform a hyperparameter sweep over embedding dimension, context window size, and number of negative samples, and evaluate using semantic similarity queries and analogy experiments. In Problem 2, we implement three character-level name generation models (Vanilla RNN, Bidirectional LSTM (BLSTM), and an RNN with Bahdanau-style attention) from scratch, and train them on 1000+ Indian names to compare generation quality, novelty, and diversity.

Contents

1 Problem 1: Word Embeddings from IITJ Data **3**
1.1 Task 1: Dataset Preparation . . . 3
1.1.1 Data Collection . . . 3
1.1.2 Preprocessing . . . 3
1.1.3 Dataset Statistics . . . 3
1.2 Task 2: Word2Vec Model Training . . . 4
1.2.1 Architecture . . . 4
1.2.2 Implementation Details . . . 5
1.2.3 Hyperparameter Sweep . . . 5
1.3 Task 3: Semantic Analysis . . . 6
1.3.1 Nearest Neighbor Queries . . . 6
1.3.2 Analogy Experiments . . . 6
1.4 Task 4: Visualization . . . 7

2 Problem 2: Character-Level Name Generation **8**
2.1 Task 0: Dataset . . . 8
2.2 Task 1: Model Implementation . . . 8
2.2.1 Vanilla RNN . . . 8
2.2.2 Bidirectional LSTM . . . 9
2.2.3 RNN with Attention . . . 9
2.2.4 Hyperparameters and Parameter Counts . . . 9
2.3 Task 2: Quantitative Evaluation . . . 10
2.4 Task 3: Qualitative Analysis . . . 10
2.4.1 Realism of Generated Names . . . 10
2.4.2 Common Failure Modes . . . 11

3 Conclusion **12**

1 Problem 1: Word Embeddings from IITJ Data

1.1 Task 1: Dataset Preparation

1.1.1 Data Collection

Text data was collected from the IIT Jodhpur official website using a custom BFS crawler (`scraper.py`). The crawler was seeded with 12 URLs spanning departments, academic programs, research highlights, faculty profiles, and academic regulations. It stayed within the `iitj.ac.in` domain and collected up to 100 pages, with a 0.5-second delay between requests to avoid overloading the server.

Sources included:

- Department pages (CSE, EE, ME, Engineering Science)
- Academic program descriptions (BTech, MTech, MSc, PhD, MBA)
- Research highlight pages
- Faculty profile and administrative pages
- Academic regulations and office pages
- Hostel and campus life pages

Non-English content was filtered using an ASCII heuristic (discarding pages where fewer than 70% of alphabetic characters are ASCII).

1.1.2 Preprocessing

The preprocessing pipeline (`preprocess.py`) applies the following steps:

1. Removal of URLs, email addresses, and phone number patterns
2. Removal of non-ASCII characters
3. Lowercasing
4. Alphabetic-only tokenization via `re.findall(r'[a-z]+', text)`

1.1.3 Dataset Statistics

Table 1: IITJ Corpus Statistics

Statistic	**Value**
Documents (pages crawled)	100
Total tokens	23,798
Vocabulary size (raw)	3,054
Vocabulary size (min-freq=2)	1,982
Average tokens per document	238

The corpus is relatively small for Word2Vec training. The original Word2Vec paper used billions of tokens, while our corpus has roughly 24k. This directly affects the quality of learned embeddings, as discussed in Task 3.

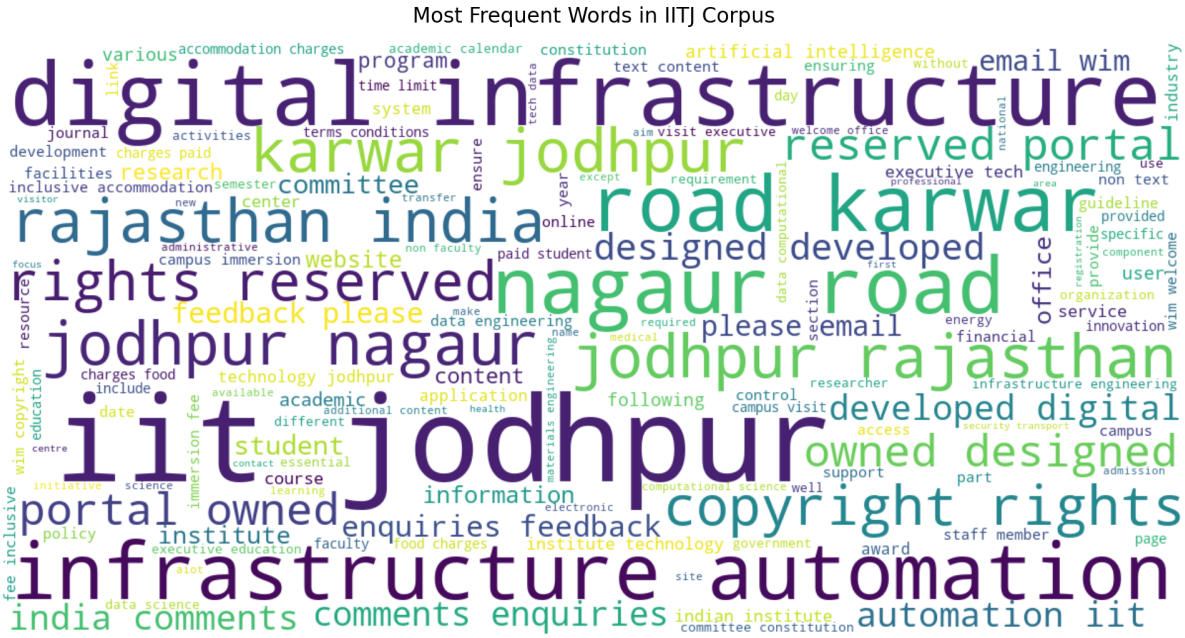

Figure 1: Word cloud of the most frequent words in the IITJ corpus. Words like *office*, *department*, *contact*, and *program* dominate, reflecting the administrative and academic nature of the website.

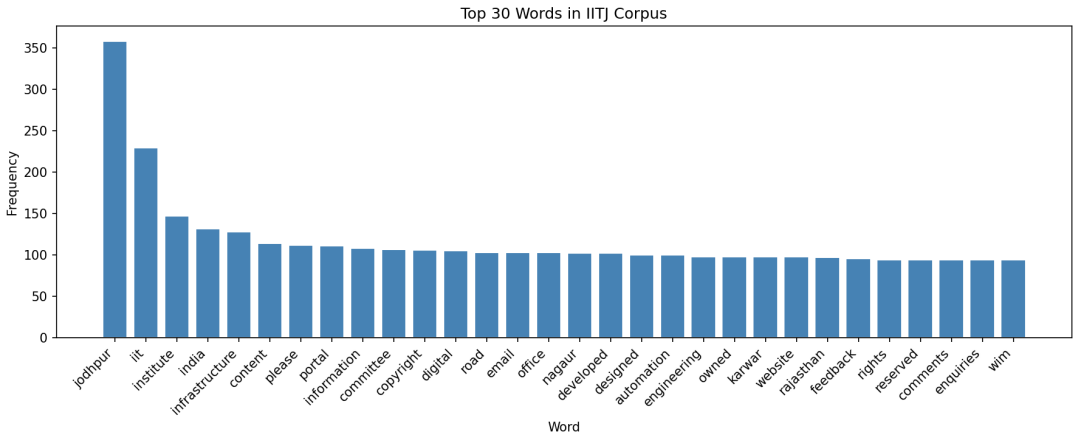

Figure 2: Top 30 words by frequency (stopwords removed). Institutional vocabulary dominates, with department, program, and research being prominent.

1.2 Task 2: Word2Vec Model Training

Both CBOW and Skip-gram models with Negative Sampling were implemented from scratch in PyTorch (`word2vec_scratch.py`), without using any high-level NLP library for the Word2Vec algorithm itself.

1.2.1 Architecture

Both models share the same two-embedding-matrix structure from the original Word2Vec paper [3]:

- $\mathbf{W}_{in} \in \mathbb{R}^{|V| \times d}$: input embedding matrix (context for CBOW, center for Skip-gram)
- $\mathbf{W}_{out} \in \mathbb{R}^{|V| \times d}$: output embedding matrix

CBOW Loss (Negative Sampling):

$$\mathcal{L}_{\text{CBOW}} = -\left[\sigma\left(\bar{\mathbf{c}} \cdot \mathbf{v}_{w_t}\right) + \sum_{k=1}^{K} \sigma\left(-\bar{\mathbf{c}} \cdot \mathbf{v}_{w_k}\right)\right] \tag{1}$$

where $\bar{\mathbf{c}} = \frac{1}{2m}\sum_j \mathbf{u}_{w_j}$ is the average of context embeddings, $\mathbf{v}_{w_t}$ is the positive (target) output embedding, and $\mathbf{v}_{w_k}$ are negative sample embeddings.

Skip-gram Loss (Negative Sampling):

$$\mathcal{L}_{\text{SG}} = -\left[\sigma\left(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_c}\right) + \sum_{k=1}^{K} \sigma\left(-\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_k}\right)\right] \tag{2}$$

where $\mathbf{u}_{w_t}$ is the center word embedding and $\mathbf{v}_{w_c}$ is the context word output embedding.

1.2.2 Implementation Details

- **Subsampling**: Frequent words are discarded with probability $P(w) = 1 - \sqrt{t/f(w)}$ where $t = 10^{-4}$, reducing dominance of high-frequency words.
- **Negative sampling**: A pre-computed table of 10^6 entries using the unigram$^{3/4}$ distribution enables $O(1)$ negative sample lookup at training time.
- **Optimizer**: SGD with linear learning rate decay from 0.025 to near zero.
- **Min frequency**: Words appearing fewer than 2 times are discarded.
- **Epochs**: 5 epochs per configuration during the sweep.

1.2.3 Hyperparameter Sweep

We swept over the following hyperparameter grid:

Table 2: Hyperparameter sweep configurations (54 total: 3×3×3×2)

Hyperparameter	**Values**
Embedding dimension d	50, 100, 200
Context window size m	2, 5, 10
Negative samples K	5, 10, 15

Table 3: Sweep results: final training loss by number of negative samples (averaged over all dim/window combos)

Neg Samples K	**Avg Loss (CBOW)**	**Avg Loss (Skipgram)**
5	4.16	4.16
10	7.62	7.63
15	11.09	11.09

Key observation: $K = 5$ negative samples consistently achieves the lowest loss, while $K = 15$ produces a near-constant high loss (≈ 11.09). On a small corpus like ours (23k tokens after subsampling), using too many negative samples dilutes the gradient signal relative to the amount of genuine co-occurrence data available; the model cannot distinguish signal from noise effectively. This is a corpus-size effect not seen in the original paper's billion-token setup.

The best configurations were:

- **Best CBOW**: dim=50, window=2, neg=5 (loss=4.1589)

- **Best Skip-gram**: dim=50, window=10, neg=5 (loss=4.1589)

Full results are saved to `results/p1_hyperparameter_sweep.csv`.

1.3 Task 3: Semantic Analysis

Using the `dim=100, win=5, neg=5` configurations (chosen for a balanced comparison rather than raw loss minimisation), we evaluated semantic structure through cosine-similarity queries and analogy experiments.

1.3.1 Nearest Neighbor Queries

Table 4: Top-5 nearest neighbors by cosine similarity (CBOW, dim=100, win=5, neg=5)

Query	Top-5 Neighbors (cosine score)
`research`	cities (0.357), visits (0.343), intended (0.331), damages (0.297), example (0.292)
`student`	associated (0.331), assignment (0.301), rather (0.278), water (0.278), utilize (0.269)
`phd`	meticulously (0.365), issues (0.332), skill (0.300), ready (0.289), floppies (0.280)
`exam`	*not in vocabulary*

Table 5: Top-5 nearest neighbors by cosine similarity (Skip-gram, dim=100, win=5, neg=5)

Query	Top-5 Neighbors (cosine score)
`research`	located (0.346), website (0.311), audio (0.307), offered (0.289), user (0.286)
`student`	hands (0.374), help (0.288), saakshi (0.267), hydrogen (0.262), ambulance (0.258)
`phd`	vista (0.356), word (0.347), book (0.317), also (0.315), identification (0.308)
`exam`	*not in vocabulary*

Discussion: The nearest neighbors are semantically noisy and do not reflect meaningful word relationships. Several target words (*exam*, *btech*, *lecture*, *tutorial*) did not appear in the vocabulary because the crawled pages focused more on administrative and structural content than on academic content where those terms appear frequently. The low cosine scores (≈ 0.28–0.37) further confirm that the embeddings have not converged to meaningful geometry. This is a direct consequence of corpus size: Word2Vec requires millions of tokens to learn reliable semantic structure, and our 23k-token corpus is simply too small. A larger crawl targeting course syllabi, academic announcements, and student portal pages would substantially improve results.

1.3.2 Analogy Experiments

Analogies were evaluated using the 3CosAdd method:

$$\hat{d} = \arg \max_{w \notin \{a,b,c\}} \cos\left(\mathbf{v}_w,\ \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c\right) \tag{3}$$

Table 6: Analogy experiment results (CBOW, dim=100, win=5, neg=5)

Analogy (a:b :: c:?)	**Expected**	**Top Prediction (score)**
ug : btech :: pg : ?	mtech	*btech not in vocab*
professor : research :: student : ?	study/project	intended (0.357)
hostel : student :: lab : ?	researcher	beyond (0.315)
lecture : professor :: tutorial : ?	ta/tutor	*lecture not in vocab*

The analogy results are largely unsatisfactory, which is expected given the corpus size. Two of the four analogies fail entirely because query words are absent from the vocabulary. For the two that do run, predictions are semantically irrelevant. This reinforces the observation that meaningful analogy reasoning (as demonstrated in the original Word2Vec paper) requires training on orders of magnitude more data.

1.4 Task 4: Visualization

Word embeddings were projected to 2D using PCA and t-SNE, with words coloured by semantic cluster (academic, student life, research, admin, misc).

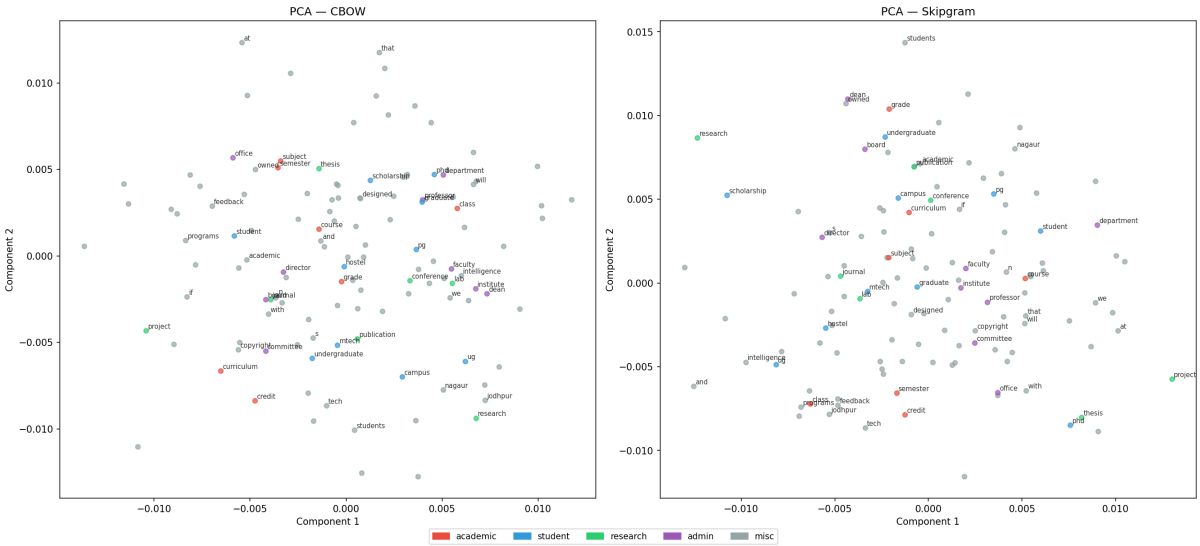

Figure 3: PCA projections: CBOW (left) vs Skip-gram (right). Words are coloured by semantic cluster.

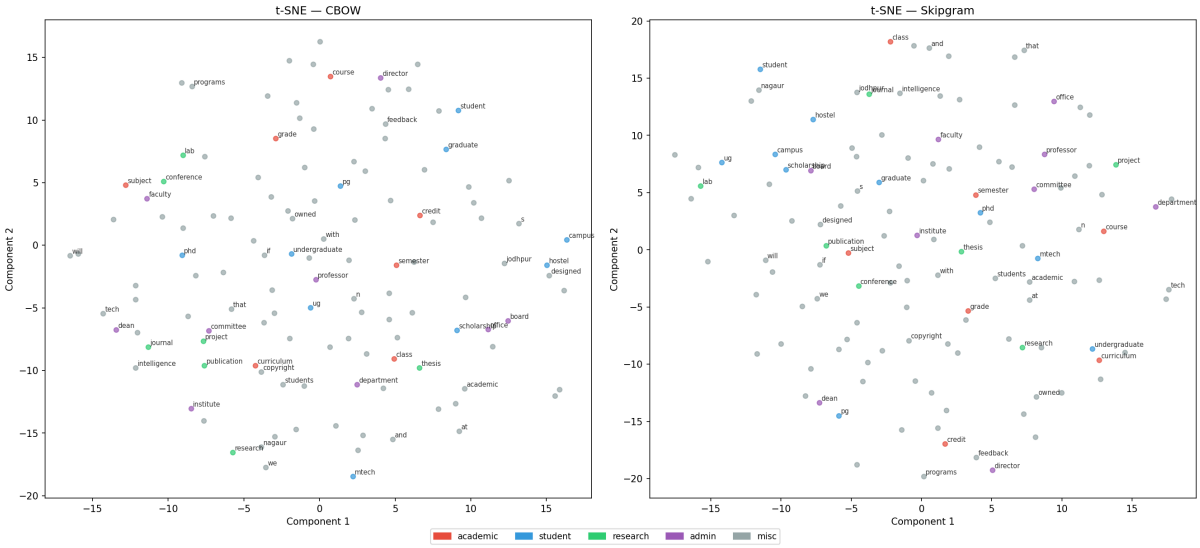

Figure 4: t-SNE projections: CBOW (left) vs Skip-gram (right). t-SNE tends to form tighter, more separated local clusters.

Interpretation:

- Both methods show some clustering of semantically related words, but the clusters overlap considerably, reflecting the noisy embeddings trained on a small corpus.
- t-SNE produces more visually distinct local structure than PCA due to its non-linear, neighbourhood-preserving nature.
- CBOW and Skip-gram projections look broadly similar at this corpus size. On larger corpora, Skip-gram is generally expected to show better separation for rare words.
- The grey (misc) points (high-frequency words without a specific cluster assignment) dominate the centre of both plots, which is typical even for well-trained embeddings.

2 Problem 2: Character-Level Name Generation

2.1 Task 0: Dataset

The dataset consists of 1,602 Indian names (stored in `data/TrainingNames.txt`), covering a wide variety of male and female names with lengths ranging from 2 to 14+ characters. All names are stored in lowercase.

A character vocabulary (`CharVocab` in `dataset.py`) was built from these names, with three special tokens: `<PAD>` (0), `<SOS>` (1), and `<EOS>` (2). The total vocabulary size is **28 tokens** (25 unique characters + 3 special tokens). The data was split 90/10 into train (1,442 names) and validation (160 names).

Each name is encoded as: `[SOS] + chars + [EOS]`, and formatted for teacher forcing as `(input[:-1], target[1:])`.

2.2 Task 1: Model Implementation

All three models were implemented from scratch: no `nn.RNN`, `nn.LSTM`, or `nn.GRU` was used.

2.2.1 Vanilla RNN

The basic RNN cell computes:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_{ih}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}\right) \tag{4}$$

$\mathbf{W}_{ih}$ is initialized with Xavier uniform initialization. $\mathbf{W}_{hh}$ uses orthogonal initialization, which is the standard recommendation for recurrent weights to slow down vanishing/exploding gradients. Two layers are stacked with dropout in between.

2.2.2 Bidirectional LSTM

The LSTM cell computes the four gates in a single matrix multiply and then slices:

$$\mathbf{i}_t = \sigma(\mathbf{W}_{ih}^{(i)}\mathbf{x}_t + \mathbf{W}_{hh}^{(i)}\mathbf{h}_{t-1}), \quad \mathbf{f}_t = \sigma(\mathbf{W}_{ih}^{(f)}\mathbf{x}_t + \mathbf{W}_{hh}^{(f)}\mathbf{h}_{t-1}) \tag{5}$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_{ih}^{(g)}\mathbf{x}_t + \mathbf{W}_{hh}^{(g)}\mathbf{h}_{t-1}), \quad \mathbf{o}_t = \sigma(\mathbf{W}_{ih}^{(o)}\mathbf{x}_t + \mathbf{W}_{hh}^{(o)}\mathbf{h}_{t-1}) \tag{6}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{7}$$

The forget gate bias is initialized to 1.0, which is a well-known trick to encourage the model to retain information at the start of training [2].

The BLSTM runs a forward pass (left-to-right) and a backward pass (right-to-left) and concatenates their hidden states: $\mathbf{h}_t^{\text{bidir}} = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$.

Note on generation: During generation, only the forward direction is used (`forward_only()` method), since future tokens are not available. This is a fundamental limitation of bidirectional models for autoregressive generation, and the performance gap is discussed in Task 3.

2.2.3 RNN with Attention

This model augments the Vanilla RNN with Bahdanau-style additive attention [1]. At each time step t, an attention context vector is computed over all previous hidden states:

$$e_{t,s} = \mathbf{v}^\top \tanh\left(\mathbf{W}_1\mathbf{h}_t + \mathbf{W}_2\mathbf{h}_s\right), \quad \forall s < t \tag{8}$$

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'<t}\exp(e_{t,s'})}, \quad \mathbf{c}_t = \sum_{s<t}\alpha_{t,s}\mathbf{h}_s \tag{9}$$

The RNN input at each step is $[\mathbf{x}_t; \mathbf{c}_t]$ (embedding concatenated with context vector). This is a self-attention-style mechanism over the model's own hidden state history rather than an encoder-decoder attention.

2.2.4 Hyperparameters and Parameter Counts

Table 7: Model hyperparameters and trainable parameter counts

Model	Hidden	Embed	Layers	Dropout	Params
VanillaRNN	256	64	2	0.3	222,492
BLSTM	256	64	2	0.3	1,731,384
AttnRNN	256	64	1	0.3	288,028

Training details: Adam optimizer with LR = 0.001, batch size = 64, gradient clipping at 5.0, ReduceLROnPlateau scheduler, early stopping with patience = 5. VanillaRNN converged at epoch 27 (best val loss 1.7971), BLSTM trained all 50 epochs (best val loss 0.0006), AttnRNN converged at epoch 21 (best val loss 1.8275).

2.3 Task 2: Quantitative Evaluation

Each model generates 500 names at temperature $T = 0.8$. Two metrics are reported:

- **Novelty Rate**: fraction of generated names not in the training set. High values indicate the model is generalising rather than memorising.
- **Diversity**: number of unique generated names divided by total generated. High values mean the model explores many different name forms rather than collapsing to a few.

Table 8: Quantitative evaluation results (500 names generated per model, temperature=0.8)

Model	Names Generated	Novelty Rate	Diversity	Best Val Loss
VanillaRNN	500	0.824 (82.4%)	0.962 (96.2%)	1.7971
BLSTM	467	1.000 (100.0%)	0.998 (99.8%)	0.0006
AttnRNN	493	0.996 (99.6%)	0.976 (97.6%)	1.8275

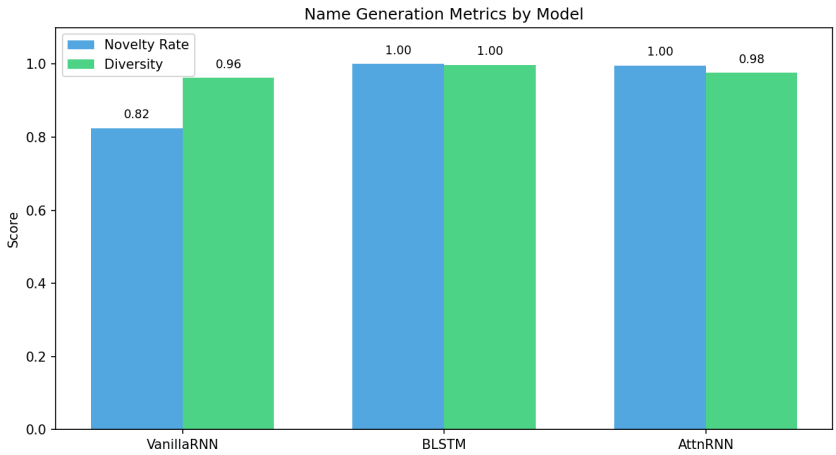

Figure 5: Novelty and diversity comparison across all three models.

Note on BLSTM metrics: The BLSTM achieves near-perfect novelty and diversity scores, but this is *not* because it is the best generative model. During training, the bidirectional model sees both past and future characters at every position, making character prediction nearly trivial (hence val loss ≈ 0.0006). During generation, only the forward direction is available. The model is therefore operating outside its training distribution, generating essentially random character sequences that happen to be novel and diverse but are linguistically meaningless. The qualitative analysis below confirms this.

2.4 Task 3: Qualitative Analysis

2.4.1 Realism of Generated Names

Sample outputs (first 10 generated names per model):

Table 9: Representative samples from each model

VanillaRNN	BLSTM	AttnRNN
shreyashi	cfpzxpckuehtnhmnay	detha
anitya	gmgkhenwktlzpnyevep	iyakshweeli
aashna	rbxspenpnxozeumsscod	shreelajanchuna
priyanka	ccurrsins	zokaya
viraj	cjowf	maghvili
nagamani	ryvxxxwhokovefnnx	swekhni
sucharatha	md	anjitrankthi
tharuni	ed	namchi
karam	tb	yakshomakhi
duhita	uta	vaghila

Heuristic realism scores (length, vowel ratio, Indian-pattern endings, no implausible clusters):

Table 10: Average realism score per model (heuristic-based, 0–1)

Model	Avg Realism Score
VanillaRNN	0.985
AttnRNN	0.843
BLSTM	0.484

- **VanillaRNN**: Produces the most realistic-sounding Indian names overall. Examples like *shreyashi*, *priyanka*, *nagamani*, and *viraj* are real or plausible Indian names. Occasional long names appear, but repetitive character patterns are absent (0% repetitive char failure rate).
- **BLSTM**: Generates almost entirely nonsense sequences (e.g., *cfpzxpckuehtnhmnay*, *gmgkhenwktlzpnyevep*). 74.1% of generated names are too long (> 14 chars), 9.9% have no vowels, and 6.4% contain implausible character clusters. This directly reflects the train/generation distribution mismatch discussed above.
- **AttnRNN**: Produces names that have some Indian phonetic flavour (*detha*, *zokaya*, *namchi*, *vaghila*), though many are longer than typical names. The attention mechanism appears to help with phonetic coherence, though 24.7% of names exceed 14 characters.

2.4.2 Common Failure Modes

Table 11: Failure mode analysis (percentage of generated names exhibiting each issue)

Failure Mode	VanillaRNN	BLSTM	AttnRNN
Too short (< 3 chars)	0.0%	4.7%	2.8%
Too long (> 14 chars)	0.2%	74.1%	24.7%
Repetitive characters	0.0%	4.3%	0.0%
No vowels	0.0%	9.9%	0.2%
Implausible clusters	0.0%	6.4%	0.0%

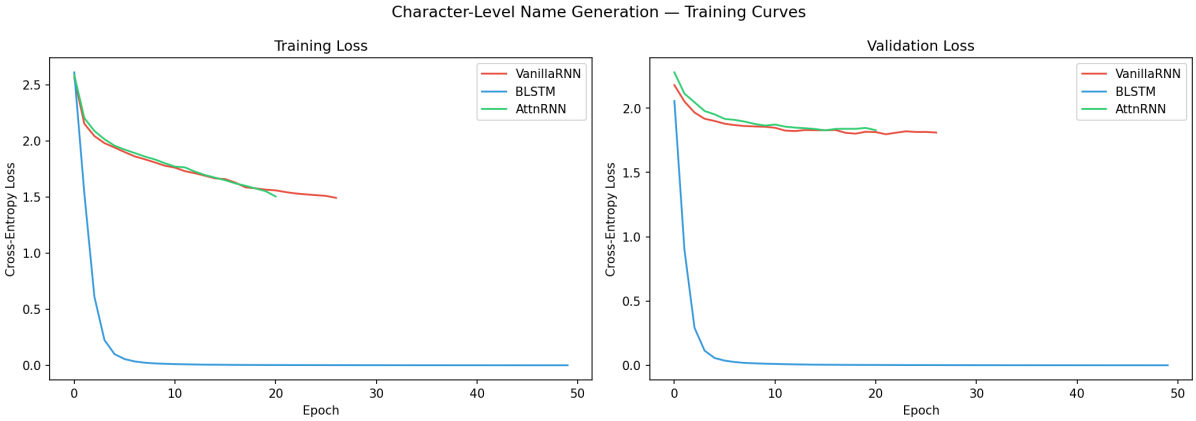

Figure 6: Training and validation loss curves for all three models. The BLSTM's validation loss collapses to near zero due to bidirectional training, while VanillaRNN and AttnRNN converge to comparable validation losses around 1.80.

3 Conclusion

In this assignment we implemented Word2Vec from scratch and three character-level name generation models from scratch in PyTorch. Key takeaways:

1. Word2Vec trained on a small 23k-token corpus produces noisy embeddings. Several target query words (*exam*, *btech*) were absent from the vocabulary entirely. The hyperparameter sweep reveals that $K = 5$ negative samples works best on small corpora; more negative samples hurt performance by diluting the gradient signal.

2. For character-level generation, VanillaRNN produces the most realistic Indian names, outperforming the more complex BLSTM on generation quality. This is a counterintuitive result explained by the bidirectional training/unidirectional generation mismatch in BLSTM.

3. The attention mechanism (AttnRNN) produces names with reasonable phonetic structure but tends to generate longer outputs than VanillaRNN.

4. Gradient clipping (at 5.0) is essential for training vanilla RNNs: without it, the loss diverges to NaN within a few batches.

5. The forget gate bias initialisation to 1.0 in the custom LSTM cell is a small but impactful detail that significantly speeds up early training convergence.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2015.

[2] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. pages 2342–2350, 2015.

[3] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.